Gustavo Martins França
Maria Eduarda Bambini

Atividade: Processamento de Dados

OPT004 - Ciência de Dados

Prof. Eduardo Henrique Monteiro Pena

Campo Mourão

Outubro / 2026

Sumário

1 Introdução . . . 3
2 Carregamento . . . 3
3 Procedimentos . . . 5
3.1 Primeira consulta . . . 5
3.1.1 DuckDB . . . 5
3.1.2 Pandas . . . 7
3.2 Segunda Consulta . . . 9
3.2.1 DuckDB . . . 10
3.2.2 Pandas . . . 12
3.3 Terceira Consulta . . . 14
3.3.1 DuckDB . . . 15
3.3.2 Pandas . . . 15
3.4 Quarta Consulta . . . 17
3.4.1 DuckDB . . . 18
3.4.2 Pandas . . . 19
3.5 Quinta consulta . . . 22
3.5.1 DuckDB . . . 22
3.5.2 Pandas . . . 23
4 Resultados . . . 24

1 Introdução

Nesta atividade, foi utilizado o dataset NYC Yellow Taxi Trip Records com o objetivo de realizar consultas analíticas sobre as corridas de táxi registradas em janeiro de 2026. Para isso, os dados, especialmente o arquivo Yellow Taxi, foram processados com as bibliotecas Pandas e DuckDB, de modo que houveram implementações de consultas em Python e SQL.

Além da implementação das consultas, a atividade também teve como objetivo comparar o tempo de leitura e de execução das operações, bem como analisar as diferenças entre desenvolver as soluções com e sem auxílio de inteligência artificial.

2 Carregamento

Para carregar o conjunto de dados, foi utilizada a função “read_parquet”, no caso do Pandas, e criação de uma tabela “corridas” no caso do DuckDB, conforme ilustrado abaixo. Após o carregamento, foram inspecionados o número de registros, as colunas disponíveis e os tipos de dados, com o objetivo de compreender a estrutura do conjunto de dados antes de realizar as consultas. No Pandas, essa inspeção foi feita por meio do método “info”, enquanto no DuckDB foi utilizada a visualização da tabela. O código para o carregamento pode ser verificado abaixo.

```
# Carregamento dos dados
import duckdb
import pandas as pd
import psutil
import time

arquivo = "yellow_tripdata_2026-01.parquet"

# Inspecionar o n mero de linhas, as colunas e os tipos de dados
df = pd.read_parquet(arquivo)

con = duckdb.connect()
con.execute(f"""
    CREATE TABLE corridas AS
    SELECT * FROM read_parquet('{arquivo}')
""")

con.table("corridas").show()

```

```
df = con.table("corridas").df()
df.info()
```

A execução do código levou 2.7 segundos e a saída retornou uma ilustração da tabela presente no arquivo, com 3.724.889 registros, como ilustrado na figura 1.

```
RangeIndex: 3724889 entries, 0 to 3724888
Data columns (total 20 columns):
 #   Column                 Dtype
---  ------                 -----
 0   VendorID               int32
 1   tpep_pickup_datetime   datetime64[us]
 2   tpep_dropoff_datetime  datetime64[us]
 3   passenger_count        Int64
 4   trip_distance          float64
 5   RatecodeID             Int64
 6   store_and_fwd_flag     str
 7   PULocationID           int32
 8   DOLocationID           int32
 9   payment_type           int64
 10  fare_amount            float64
 11  extra                  float64
 12  mta_tax                float64
 13  tip_amount             float64
 14  tolls_amount           float64
 15  improvement_surcharge  float64
 16  total_amount           float64
 17  congestion_surcharge   float64
 18  Airport_fee            float64
 19  cbd_congestion_fee     float64
```

Figura 1 – Informaçõe gerais sobre a relação `corridas`

Também foi realizado um teste comparando o tempo de carregamento entre as duas ferramentas, como ilustrado na Figura 2. É possível observar que em média a ferramenta pandas teve o tempo de carregamento em torno de 3 vezes menor.

```
Run 1: duckdb loaded data in 0.68s
Run 1: pandas loaded data in 0.21s
Run 2: duckdb loaded data in 0.62s
Run 2: pandas loaded data in 0.16s
Run 3: duckdb loaded data in 0.59s
Run 3: pandas loaded data in 0.21s

DuckDB average: 0.63
Pandas average: 0.19
```

Figura 2 – Tempo de carregamento médio de cada ferramenta

3 Procedimentos

As consultas foram implementadas, como mencionado anteriormente, tanto em Pandas quanto em SQL, utilizando o DuckDB, de modo que os resultados pudessem ser comparados. Para garantir equivalência entre as implementações, foram utilizadas os mesmos critérios para filtragem, agrupamento e contagem.

3.1 Primeira consulta

3.1.1 DuckDB

A primeira consulta levou 44 minutos para ser escrita, sendo em torno de metade desse tempo destinado à leitura da documentação e a outra metade à escrita da consulta em si. Ela teve como objetivo analisar a distribuição das corridas ao longo das horas do dia, calculando a quantidade de corridas iniciadas em cada hora, a distância média percorrida e o valor médio pago. Os resultados foram ordenados pela quantidade de corridas, em ordem decrescente.

No DuckDB, a consulta foi implementada utilizando as funções HOUR, COUNT e AVG que, respectivamente, extrai a hora de início, conta os registros e calcula as médias solicitas nos parâmetros. Em seguida, os resultados foram agrupados pela hora de início com GROUP BY e ordenados pela quantidade de corridas em ordem decrescente, utilizando o ORDER BY. Verifique a implementação abaixo.

```
1 # DuckDB
2
3 tempos_execucao = []
4 df_duckdb_hora_inicio = None
5
```

```
for i in range(3):
    inicio = time.perf_counter()

    # Consulta
    df_duckdb_hora_inicio = con.execute(f"""
        SELECT
            HOUR(tpep_pickup_datetime) AS hora_inicio_corrida,
            COUNT(*) AS num_corridas,
            AVG(trip_distance) AS distancia_media,
            AVG(total_amount) AS valor_medio_pago
        FROM read_parquet('{arquivo}')
        GROUP BY hora_inicio_corrida
        ORDER BY num_corridas DESC
    """).df()

    fim = time.perf_counter()
    tempo_decorrido = fim - inicio
    tempos_execucao.append(tempo_decorrido)

media_execucao = sum(tempos_execucao) / len(tempos_execucao)

# Exibe as m tricas de tempo e o resultado final
print(f"Tempos individuais: {[f'{t:.4f}s' for t in
   tempos_execucao]}")
print(f"M dia do tempo de execu  o: {media_execucao:.4f}
   segundos\n")

print(df_duckdb_hora_inicio)
```

```
Tempos individuais: ['0.0568s', '0.0395s', '0.0397s']
Média do tempo de execução: 0.0453 segundos

    hora_inicio_corrida  num_corridas  distancia_media  valor_medio_pago
0                    18        265574         3.191099         29.317995
1                    17        252173         6.149070         30.089539
2                    19        234542         4.336286         29.063146
3                    16        227893         4.936715         30.851314
4                    15        221677         5.255241         29.340657
5                    20        212883         5.726825         29.116929
6                    21        212544         7.311580         29.230308
7                    14        207507         6.578414         29.071705
8                    13        195708         5.526828         27.888121
9                    22        194801         5.441168         29.510664
10                   12        187435         5.295283         27.497725
11                   11        171729         7.003858         27.032998
12                   10        161419         9.127196         27.314459
13                    9        156894         7.422044         27.884764
14                   23        152622         4.027817         30.136489
15                    8        148281         8.680368         29.741648
16                    7        113463        17.346660         30.959745
17                    0        111867         4.750537         29.563454
18                    1         77821         8.247310         28.925152
19                    6         63721         9.265857         32.535776
20                    2         54105         9.167202         27.481215
21                    3         38645         9.912448         27.901107
22                    5         32912        15.211518         33.930427
23                    4         28673         7.501011         31.213174
```

Figura 3 – Captura de tela da saída da implementação da primeira consulta com DuckDB.

Após a execução do código, que levou 0.0 segundos em média, foi retornado uma tabela conforme a figura 3. Os resultados foram apresentados em ordem decrescente pela quantidade de corridas, permitindo observar que o maior volume de viagens ocorreu às 18h, com 265.574 registros, seguido das 17h, com 252.173 corridas, e das 19h, com 234.542 corridas. Em contrapartida, o menor número de viagens foi registrado às 4h, com 28.673 corridas, indicando uma concentração maior de atividade no período da tarde e início da noite.

Em relação às médias calculadas, a distância média das corridas variou entre aproximadamente 3,19 e 17,35 milhas, enquanto o valor médio pago variou entre US$ 27,03 e US$ 33,93, considerando os valores registrados na coluna “total_amount”.

3.1.2 Pandas

Enquanto no Pandas, levou-se em torno de 40 minutos para ler a documentação e em torno de 20 minutos para escrever a consulta. Foi preciso garantir inicialmente que a coluna “tpep_pickup_ datetime” estivesse no formato de data e hora, por meio da função “pd.to_datetime”, permitindo extrair a hora de cada registro com o atributo “dt.hour”.

Feito isso, os dados foram agrupados por hora, utilizando o método “groupby”, enquanto o método “agg” foi empregado para calcular a contagem de corridas e as médias de distância e valor pago. Por fim, os resultados da consulta em Pandas foram organizados

em um Data Frame e ordenados pela quantidade de corridas, com os mesmos critérios utilizados na consulta SQL. A implementação pode ser verificada abaixo.

```
# Pandas

tempos_execucao = []
df_duckdb_hora_inicio = None

for i in range(3):
    inicio = time.perf_counter()

    # Consulta
    df['tpep_pickup_datetime'] = pd.to_datetime(df['
   tpep_pickup_datetime'])

    df_pandas_hora_inicio = (
        df.groupby(df['tpep_pickup_datetime'].dt.hour)
        .agg(
                num_corridas=('tpep_pickup_datetime', 'count'),
                distancia_media=('trip_distance', 'mean'),
                valor_medio_pago=('total_amount', 'mean'),
            )
            .reset_index(names='hora_inicio_corrida')
            .sort_values('num_corridas', ascending=False)
            .reset_index(drop=True)
    )

    fim = time.perf_counter()
    tempo_decorrido = fim - inicio
    tempos_execucao.append(tempo_decorrido)

media_execucao = sum(tempos_execucao) / len(tempos_execucao)

# Exibe as m tricas de tempo e o resultado final
print(f"Tempos individuais: {[f'{t:.4f}s' for t in
   tempos_execucao]}")
print(f"M dia do tempo de execu  o: {media_execucao:.4f}
   segundos\n")

print(df_pandas_hora_inicio)
```

A execução da consulta em Pandas levou 0.1 segundos em média e apresentou os mesmos valores de quantidade de corridas, como ilustrado na figura 4, com distância média e valor médio pago para cada uma das 24 horas analisadas, mostrando a equivalência entre as duas implementações.

```
Tempos individuais: ['0.2994s', '0.1429s', '0.1441s']
Média do tempo de execução: 0.1955 segundos
```

	hora_inicio_corrida	num_corridas	distancia_media	valor_medio_pago
0	18	265574	3.191099	29.317995
1	17	252173	6.149070	30.089539
2	19	234542	4.336286	29.063146
3	16	227893	4.936715	30.851314
4	15	221677	5.255241	29.340657
5	20	212883	5.726825	29.116929
6	21	212544	7.311580	29.230308
7	14	207507	6.578414	29.071705
8	13	195708	5.526828	27.888121
9	22	194801	5.441168	29.510664
10	12	187435	5.295283	27.497725
11	11	171729	7.003858	27.032998
12	10	161419	9.127196	27.314459
13	9	156894	7.422044	27.884764
14	23	152622	4.027817	30.136489
15	8	148281	8.680368	29.741648
16	7	113463	17.346660	30.959745
17	0	111867	4.750537	29.563454
18	1	77821	8.247310	28.925152
19	6	63721	9.265857	32.535776
20	2	54105	9.167202	27.481215
21	3	38645	9.912448	27.901107
22	5	32912	15.211518	33.930427
23	4	28673	7.501011	31.213174

Figura 4 – Captura de tela da saída da implementação da primeira consulta com Pandas.

3.2 Segunda Consulta

Na segunda consulta, foi utilizada a ferramenta de inteligência artificial Claude para implementar uma análise que identifica pares de corridas distintas que apresentassem a mesma origem e o mesmo destino, com horários de início separados por, no máximo, cinco minutos, considerando apenas as viagens iniciadas em 15 de janeiro de 2026, entre 8h e 9h.

Dessa forma, foi escrito um prompt para que a IA gerasse o código da consulta, e a ferramenta demorou menos de 1 minuto para retornar o código. No total, a conversa com a ferramenta levou em torno de 8 minutos e as modificações no código para adição do cálculo do tempo de execução levou em torno de 2 minutos. O prompt inserido na ferramenta Claude foi o seguinte: “Realize a seguinte consulta "Encontre pares de corridas distintas que tenham a mesma origem (PULocationID), o mesmo destino (DOLocationID) e horários de início (tpep_pickup_datetime) com diferença de, no máximo, cinco minutos. Conte cada par apenas uma vez e apresente somente a quantidade

total de pares. Para limitar o volume, considere as corridas iniciadas em 15/01/2026, das 8h às 9h (sem incluir 9h).", no conjunto de dados Yellow Taxi, janeiro de 2026: https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2026-01.parquet. Faça uma implementação com Pandas e outra com DuckDB (em blocos de código diferentes), as duas devem ter os mesmos resultados (outputs). Repita cada medição três vezes e informe a média da leitura e a média da execução em cada ferramenta. O conjunto de dados já foi carregado".

A ferramenta forneceu duas implementações equivalentes, utilizando o método merge no Pandas para combinar as corridas com a mesma origem e destino, e uma operação JOIN no DuckDB para relacionar os registros que atendiam aos mesmos critérios. Nas duas implementações, foi utilizado um identificador para cada corrida, evitando que uma viagem fosse comparada consigo mesma e que um mesmo par fosse contabilizado duas vezes, enquanto a diferença entre os horários de início foi limitada a cinco minutos. Verifique os trechos de código e suas saídas abaixo.

3.2.1 DuckDB

Na implementação com DuckDB, foi realizada uma alteração na linha "`total = con_teste.execute(consulta_duckdb).fetchone()[0]  type: ignore`", adicionando o " type: ignore" para ignorar um aviso do Pylance.

```
1  # DuckDB
2
3  import time
4  import duckdb
5
6  arquivo = "yellow_tripdata_2026-01.parquet"
7
8  consulta_duckdb = """
9      WITH janela AS (
10         SELECT
11             rowid AS id,
12             PULocationID,
13             DOLocationID,
14             tpep_pickup_datetime
15         FROM corridas
16         WHERE tpep_pickup_datetime >= TIMESTAMP '2026-01-15
       08:00:00'
17           AND tpep_pickup_datetime <  TIMESTAMP '2026-01-15
       09:00:00'
```

```
    )
    SELECT COUNT(*) AS total_pares
    FROM janela a
    JOIN janela b
      ON  a.PULocationID = b.PULocationID
      AND a.DOLocationID = b.DOLocationID
      AND a.id < b.id
      AND b.tpep_pickup_datetime BETWEEN a.tpep_pickup_datetime -
    INTERVAL 5 MINUTE
                                    AND a.tpep_pickup_datetime +
    INTERVAL 5 MINUTE
"""

repeticoes = 3
tempos_leitura_db = []
tempos_execucao_db = []
resultados_db = []

con_teste = duckdb.connect()

for i in range(repeticoes):
    # Leitura (carrega o parquet em uma tabela)
    t0 = time.perf_counter()
    con_teste.execute(f"""
        CREATE OR REPLACE TABLE corridas AS
        SELECT * FROM read_parquet('{arquivo}')
    """)
    t1 = time.perf_counter()

    # Execu  o
    total = con_teste.execute(consulta_duckdb).fetchone()[0] #
   type: ignore
    t2 = time.perf_counter()

    tempos_leitura_db.append(t1 - t0)
    tempos_execucao_db.append(t2 - t1)
    resultados_db.append(total)
    print(f"[DuckDB] repeti  o {i + 1}: leitura={t1 - t0:.4f}s
   | execu  o={t2 - t1:.4f}s | pares={total}")

media_leitura_db = sum(tempos_leitura_db) / repeticoes
media_execucao_db = sum(tempos_execucao_db) / repeticoes
```

```
56
57 print(f"\n[DuckDB] Total de pares: {resultados_db[0]}")
58 print(f"[DuckDB] M dia da leitura:   {media_leitura_db:.4f} s")
59 print(f"[DuckDB] M dia da execu  o:  {media_execucao_db:.4f} s
   ")
```

```
[DuckDB] repetição 1: leitura=0.5116s | execução=0.0032s | pares=3721
[DuckDB] repetição 2: leitura=1.2091s | execução=0.0174s | pares=3721
[DuckDB] repetição 3: leitura=0.7850s | execução=0.0032s | pares=3721

[DuckDB] Total de pares: 3721
[DuckDB] Média da leitura:   0.8352 s
[DuckDB] Média da execução:  0.0080 s
```

Figura 5 – Captura de tela da saída da implementação da segunda consulta com DuckDB.

3.2.2 Pandas

```
1  # Pandas
2
3  import time
4  import pandas as pd
5
6  arquivo = "yellow_tripdata_2026-01.parquet"
7
8  inicio = pd.Timestamp("2026-01-15 08:00:00")
9  fim = pd.Timestamp("2026-01-15 09:00:00")
10 limite = pd.Timedelta(minutes=5)
11
12
13 def consulta_pandas(df):
14     # 1) Filtra a janela de tempo (in cio inclusive, fim
       exclusivo)
15     mascara = (df["tpep_pickup_datetime"] >= inicio) & (df["
       tpep_pickup_datetime"] < fim)
16     janela = df.loc[mascara, ["PULocationID", "DOLocationID", "
       tpep_pickup_datetime"]]
17
18     # 2) Guarda a posi  o original para identificar cada
       corrida
19     janela = janela.reset_index().rename(columns={"index": "id"})
20
21     # 3) Auto-jun  o: mesma origem e mesmo destino
22     juntas = janela.merge(
```

```
        janela,
        on=["PULocationID", "DOLocationID"],
        suffixes=("_a", "_b"),
    )

    # 4) id_a < id_b: corridas distintas e cada par contado uma
    nica  vez
    juntas = juntas[juntas["id_a"] < juntas["id_b"]]

    # 5) Diferen a de hor rio de no m ximo 5 minutos (
   inclusivo)
    diferenca = (juntas["tpep_pickup_datetime_a"] - juntas["
   tpep_pickup_datetime_b"]).abs()
    return int((diferenca <= limite).sum())


repeticoes = 3
tempos_leitura_pd = []
tempos_execucao_pd = []
resultados_pd = []

for i in range(repeticoes):
    # Leitura
    t0 = time.perf_counter()
    df_pd = pd.read_parquet(arquivo)
    t1 = time.perf_counter()

    # Execu  o
    total = consulta_pandas(df_pd)
    t2 = time.perf_counter()

    tempos_leitura_pd.append(t1 - t0)
    tempos_execucao_pd.append(t2 - t1)
    resultados_pd.append(total)
    print(f"[Pandas] repeti  o {i + 1}: leitura={t1 - t0:.4f}s
   | execu  o={t2 - t1:.4f}s | pares={total}")

media_leitura_pd = sum(tempos_leitura_pd) / repeticoes
media_execucao_pd = sum(tempos_execucao_pd) / repeticoes

print(f"\n[Pandas] Total de pares: {resultados_pd[0]}")
print(f"[Pandas] M dia da leitura:   {media_leitura_pd:.4f} s")
```

```
61 print(f"[Pandas] M dia da execu  o:  {media_execucao_pd:.4f} s
      ")
```

```
[Pandas] repetição 1: leitura=0.3395s | execução=0.0281s | pares=3721
[Pandas] repetição 2: leitura=0.3202s | execução=0.0241s | pares=3721
[Pandas] repetição 3: leitura=0.2974s | execução=0.0252s | pares=3721

[Pandas] Total de pares: 3721
[Pandas] Média da leitura:   0.3190 s
[Pandas] Média da execução:  0.0258 s
```

Figura 6 – Captura de tela da saída da implementação da segunda consulta com Pandas.

Após a execução dos códigos, levando 0.0 segundos em ambas as ferramentas, elas retornaram o valor 3.721 para pares de corridas, conforme ilustrado na figura 7, indicando que os resultados estavam de acordo com o esperado e eram equivalentes entre si.

```
1 assert len(set(resultados_pd + resultados_db)) == 1, "Os
     resultados s o diferentes!"
2
3 print(f"Total de pares (Pandas = DuckDB): {resultados_pd[0]}")
4 print(f"\n{'Ferramenta':<10} {'M dia leitura (s)':>20} {'M dia
     execu  o (s)':>20}")
5 print(f"{'Pandas':<10} {media_leitura_pd:>20.4f} {
     media_execucao_pd:>20.4f}")
6 print(f"{'DuckDB':<10} {media_leitura_db:>20.4f} {
     media_execucao_db:>20.4f}")
```

Listing 1 – Código gerado pela IA Claude para comparação das duas implementações.

```
Total de pares (Pandas = DuckDB): 3721

Ferramenta    Média leitura (s)   Média execução (s)
Pandas                   0.3190               0.0258
DuckDB                   0.8352               0.0080
```

Figura 7 – Captura de tela da saída do desempenho geral da segunda questão.

3.3 Terceira Consulta

A terceira consulta teve como objetivo identificar quais destinos das corridas têm valor e distância médios maiores por corrida.

3.3.1 DuckDB

Foi realizada uma consulta de modo que a média dos valores das corridas desconsiderasse gorjetas. Também foram descartados registros com valores negativos de `total_amount - tip_amount` e `trip_distance`, como ilustrado no trecho de código abaixo.

```
    SELECT
        DOLocationID AS locationID,
        AVG(total_amount - tip_amount) AS avg_price,
        AVG(trip_distance) AS avg_distance
    FROM corridas
    WHERE
        total_amount - tip_amount > 0
        AND trip_distance > 0
    GROUP BY locationID
    ORDER BY avg_price DESC, avg_distance DESC
```

O tempo execução médio e os resultados estão ilustrados na figura 8.

```
Run 1: done in 0.04s
Run 2: done in 0.04s
Run 3: done in 0.04s
Average exec time: 0.04s

     locationID   avg_price  avg_distance
0            44  144.847826     28.634783
1           204  132.146000     27.223333
2             5  124.294615     25.486923
3            84  120.947037     26.350000
4             1  118.107933     17.519040
..          ...         ...           ...
255         142   21.524742      2.667030
256         141   21.478164      3.160321
257          43   21.342153      2.188403
258         236   20.254914      3.416549
259         237   19.674754      2.709077
```

Figura 8 – Saída da implementação da terceira consulta com DuckDB.

No total, foram necessários 10 minutos para a idealização da consulta e 30 minutos para a implementação.

3.3.2 Pandas

Enquanto na implementação com Pandas, foram gastos em torno de 10 minutos lendo a documentação do Pandas e 20 minutos escrevendo a consulta. Inicialmente na consulta, foram selecionadas apenas as corridas com valor total superior ao valor da

gorjeta e distância percorrida positiva. Feito isso, calculou-se o preço da corrida sem a gorjeta, subtraindo o valor da gorjeta do valor total pago, em que para cada local de destino, foram calculadas a média dos preços das corridas e a distância média percorrida. Por fim, os resultados foram ordenados pelo preço médio em ordem decrescente e, em caso de empate, pela distância média, também em ordem decrescente.

A implementação utiliza o método "query" para filtrar os registros e uma nova coluna para armazenar o preço sem a gorjeta. A função "groupby", combinada com "agg", permite agrupar os dados pelo identificador do local de destino (DOLocationID) e calcular as médias solicitadas. Em seguida, o método "reset_index" reorganiza o índice do resultado, enquanto "rename" altera o nome da coluna de identificação para "locationID". Por fim, "sort_values" realiza a ordenação dos resultados de acordo com os critérios definidos.

```
# Pandas

tempos_execucao = []
df_duckdb_hora_inicio = None

for i in range(3):
    inicio = time.perf_counter()

    df_filtered = df.query("total_amount - tip_amount > 0 and trip_distance > 0").copy()
    df_filtered["price"] = df_filtered["total_amount"] - df_filtered["tip_amount"]

    df_pandas_media_destinos = (
        df_filtered.groupby("DOLocationID")
        .agg(
            avg_price=("price", "mean"),
            avg_distance=("trip_distance", "mean")
        )
        .reset_index()
        .rename(columns={"DOLocationID": "locationID"})
        .sort_values(by=["avg_price", "avg_distance"], ascending=[False, False])
        .reset_index(drop=True)
    )

    fim = time.perf_counter()
    tempo_decorrido = fim - inicio
```

```
26      tempos_execucao.append(tempo_decorrido)
27
28  media_execucao = sum(tempos_execucao) / len(tempos_execucao)
29
30  # Exibe as m tricas de tempo e o resultado final
31  print(f"Tempos individuais: {[f'{t:.4f}s' for t in
        tempos_execucao]}")
32  print(f"M dia do tempo de execu  o: {media_execucao:.4f}
        segundos\n")
33
34  print(df_pandas_media_destinos)
```

O tempo médio de execução foi 0.39 segundos, como pode ser verificado na figura 9, consideravelmente maior que a implementação utilizando DuckDB.

```
Tempos individuais: ['0.4351s', '0.3868s', '0.3632s']
Média do tempo de execução: 0.3950 segundos

    hora_inicio_corrida  num_corridas  distancia_media  valor_medio_pago
0                    18        265574         3.191099         29.317995
1                    17        252173         6.149070         30.089539
2                    19        234542         4.336286         29.063146
3                    16        227893         4.936715         30.851314
4                    15        221677         5.255241         29.340657
5                    20        212883         5.726825         29.116929
6                    21        212544         7.311580         29.230308
7                    14        207507         6.578414         29.071705
8                    13        195708         5.526828         27.888121
9                    22        194801         5.441168         29.510664
10                   12        187435         5.295283         27.497725
11                   11        171729         7.003858         27.032998
12                   10        161419         9.127196         27.314459
13                    9        156894         7.422044         27.884764
14                   23        152622         4.027817         30.136489
15                    8        148281         8.680368         29.741648
16                    7        113463        17.346660         30.959745
17                    0        111867         4.750537         29.563454
18                    1         77821         8.247310         28.925152
19                    6         63721         9.265857         32.535776
20                    2         54105         9.167202         27.481215
21                    3         38645         9.912448         27.901107
22                    5         32912        15.211518         33.930427
23                    4         28673         7.501011         31.213174
```

Figura 9 – Saída da implementação da terceira consulta com Pandas.

3.4 Quarta Consulta

Na quarta consulta, foi utilizada a ferramenta de inteligência artificial Claude para implementar uma análise que identifica a duração média das corridas por hora ao

longo do dia. Para cada implementação, com DuckDB e Pandas, foram utilizados prompts diferentes.

3.4.1 DuckDB

O prompt utilizado foi o seguinte: “Realize a seguinte consulta: "Como a duração média das corridas varia ao longo do dia?". Utilize o DuckDB em python. Considerede que o conjunto de dados já está carregado em memória: "df = pd.read_parquet(arquivo) con = duckdb.connect() con.execute(f"CREATE TABLE corridas AS SELECT * FROM read_parquet('arquivo') ") "e que as colunas disponívies no conjunto de dados são "# 0 VendorID int32 # 1 tpep_pickup_datetime datetime64[us] # 2 tpep_dropoff_datetime datetime64[us] # 3 passenger_count Int64 # 4 trip_distance float64 # 5 RatecodeID Int64 # 6 store_and_fwd_flag str # 7 PULocationID int32 # 8 DOLocationID int32 # 9 payment_type int64 # 10 fare_amount float64 # 11 extra float64 # 12 mta_tax float64 # 13 tip_amount float64 # 14 tolls_amount float64 # 15 improvement_surcharge float64 # 16 total_amount float64 # 17 congestion_surcharge float64 # 18 Airport_fee float64 # 19 cbd_congestion_fee float64 "”.

A consulta gerada pela ferramenta está ilustrada no trecho de código abaixo. O tempo de carregamento e execução médios e os resultados da implementação estão ilustrados na figura 10.

```
1   SELECT
2        EXTRACT(hour FROM tpep_pickup_datetime)::INT AS hora,
3        ROUND(AVG(
4            date_diff('microsecond', tpep_pickup_datetime,
    tpep_dropoff_datetime)
5            / 60000000.0
6        ), 2) AS duracao_media_min,
7        COUNT(*) AS total_corridas
8    FROM corridas
9    WHERE tpep_dropoff_datetime > tpep_pickup_datetime
10   GROUP BY hora
11   ORDER BY hora
```

```
Average load time: 0.34s
Average exec time: 0.01s
    hora  duracao_media_min  total_corridas
0      0              15.10          110909
1      1              14.23           77097
2      2              13.78           53628
3      3              13.88           38338
4      4              15.90           28487
5      5              19.66           32736
6      6              21.26           63156
7      7              20.06          112085
8      8              18.99          146233
9      9              18.19          154631
10    10              17.83          159081
11    11              17.70          169330
12    12              17.86          184917
13    13              17.97          192894
14    14              19.09          204535
15    15              19.50          218623
16    16              19.73          224961
17    17              18.41          249046
18    18              16.75          262290
19    19              15.97          231785
20    20              15.68          210617
21    21              15.27          210242
22    22              15.46          192884
23    23              15.44          151314
```

Figura 10 – Saída da implementação da quarta consulta com DuckDB.

No total, foram necessários 4 minutos para escrever o prompt, 5 para validar a resposta e 2 para adicionar a contagem de tempo à implementação gerada.

3.4.2 Pandas

Já para a implementação em Pandas, o seguinte prompt foi submetido à ferramenta: “Realize a seguinte consulta "Como a duração média das corridas varia ao longo do dia? ", no conjunto de dados Yellow Taxi, janeiro de 2026: https://d37ci6vzurychx.cloudfront.net /trip-data/yellow_tripdata_2026-01.parquet. Faça uma implementação com Pandas. Repita cada medição três vezes e informe a média da leitura e a média da execução em cada ferramenta. O conjunto de dados já foi carregado”.

Na conversa com a IA, escrevendo o prompt e validando a resposta, gastou-se em torno de 5 minutos e a adição da contagem do tempo de execução no código levou em torno de 1 minuto. O código resultado encontra-se abaixo.

```
1 # Pandas
2
3 tempos_execucao = []
4 df_duckdb_hora_inicio = None
```

```
5
6
7  for i in range(3):
8      inicio = time.perf_counter()
9
10     resultado_pandas = pd.DataFrame()
11
12     # 1) Mant m apenas corridas com dura  o positiva (igual ao
       WHERE do DuckDB)
13     mascara = df["tpep_dropoff_datetime"] > df["
       tpep_pickup_datetime"]
14     corridas_validas = df[mascara].copy()
15
16     # 2) Calcula a dura  o em minutos e a hora de in cio da
       corrida
17     duracao = corridas_validas["tpep_dropoff_datetime"] -
       corridas_validas["tpep_pickup_datetime"]
18     corridas_validas["duracao_min"] = duracao.dt.total_seconds()
       / 60
19     corridas_validas["hora"] = corridas_validas["
       tpep_pickup_datetime"].dt.hour
20
21     # 3) Agrupa por hora: m dia da dura  o e quantidade de
       corridas
22     agrupado = corridas_validas.groupby("hora").agg(
23         duracao_media_min=("duracao_min", "mean"),
24         total_corridas=("duracao_min", "count"),
25     )
26
27     # 4) Arredonda e transforma o  ndice  (hora) em coluna, j
       ordenado por hora
28     agrupado["duracao_media_min"] = agrupado["duracao_media_min"
       ].round(2)
29     resultado_pandas = agrupado.reset_index()
30
31     fim = time.perf_counter()
32     tempo_decorrido = fim - inicio
33     tempos_execucao.append(tempo_decorrido)
34
35 media_execucao = sum(tempos_execucao) / len(tempos_execucao)
36
37 # Exibe as m tricas de tempo e o resultado final
```

```
38 print(f"Tempos individuais: {[f'{t:.4f}s' for t in
       tempos_execucao]}")
39 print(f"M dia do tempo de execu  o: {media_execucao:.4f}
       segundos\n")
40
41 print(resultado_pandas)
```

Note que a implementação utiliza uma máscara booleana para filtrar as corridas válidas e operações com datas para calcular suas durações em minutos. O atributo "dt.hour" permite extrair a hora de início de cada viagem, enquanto os métodos "groupby" e "agg" agrupam os registros por hora e calculam a duração média e a quantidade de corridas. Em seguida, o método "round" arredonda os valores médios, e "reset_index" transforma a hora, utilizada como índice do agrupamento, em uma coluna do resultado.

A única alteração realizada no código gerado, foi a adição do cálculo do tempo de execução, também utilizado em outras consultas. Desse modo, o tempo médio de execução obtido foi de 0.4 segundos, como é possível notar na figura 11.

```
Tempos individuais: ['0.5091s', '0.4793s', '0.4504s']
Média do tempo de execução: 0.4796 segundos

    hora  duracao_media_min  total_corridas
0      0              15.10          110909
1      1              14.23           77097
2      2              13.78           53628
3      3              13.88           38338
4      4              15.90           28487
5      5              19.66           32736
6      6              21.26           63156
7      7              20.06          112085
8      8              18.99          146233
9      9              18.19          154631
10    10              17.83          159081
11    11              17.70          169330
12    12              17.86          184917
13    13              17.97          192894
14    14              19.09          204535
15    15              19.50          218623
16    16              19.73          224961
17    17              18.41          249046
18    18              16.75          262290
19    19              15.97          231785
20    20              15.68          210617
21    21              15.27          210242
22    22              15.46          192884
23    23              15.44          151314
```

Figura 11 – Saída da implementação da quarta consulta com pandas.

3.5 Quinta consulta

A quinta e última consulta teve como objetivo identificar a quantidade pares distintos de corridas com período de viagem sobrepostos. Foram consideradas corridas com início em 15/01/2026 às 08:00 e as janelas de tempo 08:00 - 09:00, 08:00 - 10:00 e 08:00 - 12:00.

3.5.1 DuckDB

Foi realizado um self join na relação `corridas`, de modo que cada par de corridas (c1, c2) com a data-hora do início da corrida de c1 fosse menor que a data-hora do fim da corrida de c2 e vice-versa, como ilustrado no trecho de código abaixo. Para a contagem dos pares únicos, foi utilizado o operador de comparação `<`, evitando a contagem duplicada de um par. Como não há um identificador de corridas na relação `corridas`, foi utilizada a pseudocoluna `rowid`[1] para garantir a unicidade de cada corrida sem introduzir novos atributos na relação ou criar identificadores em tempo de execução.

```
1   SELECT
2       COUNT(*) AS unique_pairs_count
3   FROM (SELECT
4           c1.rowid AS tpep1,
5           c2.rowid AS tpep2
6       FROM corridas c1, corridas c2
7       WHERE
8           c1.tpep_pickup_datetime >= '2026-01-15 {t_start}'
9           AND c2.tpep_pickup_datetime >= '2026-01-15 {t_start}'
10          AND c1.tpep_pickup_datetime < '2026-01-15 {t_end}'
11          AND c2.tpep_pickup_datetime < '2026-01-15 {t_end}'
12          AND c1.tpep_pickup_datetime < c2.
    tpep_dropoff_datetime
13          AND c2.tpep_pickup_datetime < c1.
    tpep_dropoff_datetime)
14   WHERE tpep1 < tpep2
```

O tempo de carregamento e execução médios de cada iteração está ilustrado na figura 12. O aumento na quantidade de pares únicos entre janelas de tempo são justificados pela quantidade de corridas realizadas próximas aos horários de pico 08:00 e 12:00.

[1] Detalhado na documentação do DuckDB

```
Run 1: 08:00:00 to 09:00:00 -  9907081 unique pairs in 0.53s
Run 1: 08:00:00 to 10:00:00 - 23145124 unique pairs in 1.62s
Run 1: 08:00:00 to 12:00:00 - 46129301 unique pairs in 5.03s
Run 1: Average exec time: 2.4s

Run 2: 08:00:00 to 09:00:00 -  9907081 unique pairs in 0.49s
Run 2: 08:00:00 to 10:00:00 - 23145124 unique pairs in 1.63s
Run 2: 08:00:00 to 12:00:00 - 46129301 unique pairs in 5.72s
Run 2: Average exec time: 2.61s

Run 3: 08:00:00 to 09:00:00 -  9907081 unique pairs in 0.44s
Run 3: 08:00:00 to 10:00:00 - 23145124 unique pairs in 1.58s
Run 3: 08:00:00 to 12:00:00 - 46129301 unique pairs in 5.98s
Run 3: Average exec time: 2.67s
```

Figura 12 – Saída da implementação da quinta consulta com DuckDB

No total, foram necessários 30 minutos para revisar a sintaxe da linguagem SQL, 20 minutos lendo a documentação do DuckDB e conversas no Stack Overflow, e 50 minutos para a implementação do programa.

3.5.2 Pandas

Foi realizado um *cross join* num objeto `pd.DataFrame` contendo apenas os registros em que as horas-data de início e fim estejam contidas na janela de tempo de uma iteração do algoritmo. A relação resultante é análoga à descrita na seção 3.5.1.

Devido à quantidade de registros em um produto cartesiano (n^2 registros), foi implementado um fluxo de finalização caso a memória necessária para efetuar a operação não esteja disponível, como ilustrado no trecho de código abaixo. Além disso, o carregamento do conjunto de dados foi realizado de modo que apenas as colunas necessárias para a consulta fossem carregadas em memória, entretanto ainda não foi possível obter resultados para as janelas de tempo 08:00 - 10:00 e 08:00 - 12:00.

```
1  ...
2  df = pd.read_parquet(arquivo, columns=["tpep_pickup_datetime", "
       tpep_dropoff_datetime"])
3  ...
4  df_tpep = df.query(
5              f"tpep_pickup_datetime >= '2026-01-15 {t_start}' "
6              f"and tpep_pickup_datetime < '2026-01-15 {t_end}'"
7          ).reset_index(names="tpep_id")
8
9          df_tpep_len = len(df_tpep)
10
```

```
11          if df_tpep_len * df_tpep_len * BYTES_ROW > psutil.
    virtual_memory().available:
12              memory_exceeded = True
13          else:
14              try:
15                  df_pairs = df_tpep.merge(df_tpep, how="cross",
    suffixes=("_1", "_2")).query(
16                      "tpep_id_1 < tpep_id_2 "
17                      "and tpep_pickup_datetime_1 <
    tpep_dropoff_datetime_2 "
18                      "and tpep_pickup_datetime_2 <
    tpep_dropoff_datetime_1"
19                  )
20
21                  pairs_count = len(df_pairs)
```

A variável `BYTES_ROW` é uma estimativa para a quantidade de bytes de uma linha de um produto cartesiano do objeto `df_tpep`, e é dada por $6 * 8$: 6 colunas (3 colunas presentes em cada lado do *join*) e 8 bytes (tamanho dos índices e timestamps).

O tempo de carregamento e execução médios de cada iteração bem sucedida estão ilustrados na figura 13. É possível notar que a execução do algoritmo para a segunda janela de tempo (08:00 - 10:00) é suficiente para exaurir a memória disponível na máquina de testes, e portanto a execução foi finalizada. Entretanto, é possível notar a equivalência do número de pares encontrados com a implementação utilizando o DuckDB.

```
Run 1: 08:00:00 to 09:00:00 -  9907081 unique pairs in 3.49s
Run 1: 08:00:00 to 10:00:00 exceeded available memory
Further windows will not be considered; stopped at run 1
```

Figura 13 – Saída da implementação da quinta consulta com Pandas

No total, foram necessários 15 minutos para ler a documentação dos métodos `pd.read_parquet()` e `df.merge()`, 35 minutos para a implementação do programa e 40 minutos para evitar que uma consulta consuma toda a memória disponível no sistema.

4 Resultados

Conforme a execução das consultas, foi notório um crescimento da diferença entre o tempo de execução das implementações em DuckDB e Pandas. Na primeira e segunda questão, não houve nenhuma diferença significativa no tempo de execução das duas im-

plementações. Na terceira e quarta questão já se nota uma diferença maior, com em torno de 1.5 segundos a mais na implementação com Pandas.

É notável que, em todas as consultas, as implementações com DuckDB permaneceram com um tempo de execução constante de 0.0 segundos. Além disso, uma facilidade encontrada ao utilizar a ferramenta DuckDB foi sua sintaxe similar a uma consulta comum em um banco de dados, descartando a necessidade de aprender uma nova sintaxe para escrever uma consulta.

Com a pergunta 5, foi possível observar que o tempo de execução da consulta com a ferramenta DuckDB aumentou mais rapidamente que o tamanho da janela de tempo. Na figura 12, o tempo de execução para a janela de tempo 08:00 - 09:00 foi aproximadamente 0,2s e 9.907.081 pares únicos foram identificados, enquanto par a janela de tempo 08:00 - 10:00 foi aproximadamente 0,83s e 23.145.124 pares únicos foram identificados.

A quantidade de pares aumentou em aproximadamente 2 vezes, e o tempo de execução aumentou em aproximadamente 4 vezes. O mesmo comportamento foi observado entre as janelas de tempo 08:00 - 10:00 e 08:00 - 12:00: os pares únicos identificados foram 23.145.124 e 46.129.301, enquanto os tempos de execução foram aproximadamente 0,85s e 2,7s, respectivamente.

Além disso, a maior diferença entre implementar as questões com ou sem auxílio da ferramenta de inteligência artificial foi o tempo de escrita das consultas, visto que a ferramenta gerou as consultas quase instantaneamente, enquanto as consultas escritas manualmente demoraram, em média, 54 minutos.

Os códigos gerados pelo Claude AI não precisaram de correção no sentido semântico, pois as consultas implementadas funcionaram corretamente. De modo que apenas alguns ajustes para adaptar o código ao Jupyter Notebook foram realizados.